

# React Internals

*A Complete Guide to How React Works Under the Hood*

From the Stack Reconciler to Passive Effects

---

## Introduction

Most React developers can build components, manage state, and wire up effects, but few understand what React is actually doing beneath the surface.

- Why does a deeply nested component tree sometimes make your UI feel sluggish?
- Why does `useEffect` fire after the browser paints, but `useLayoutEffect` fires before?
- Why does `React.memo` skip a re-render in one situation but not another?

The answers to all of these questions live in React's internals specifically, in the Fiber architecture that was introduced in React 16 and has been the foundation of every major React feature since, including Concurrent Mode, Suspense, `startTransition`, and the React Compiler introduced in React 19.

This guide takes you on a complete journey through React's rendering engine. We start with

- why the original Stack Reconciler had to be replaced
- move through every stage of the Fiber architecture
- and end with a deep understanding of the Commit Phase and how effects are run.

By the end, you'll have a mental model that makes React's behaviour feel predictable and not magical.

---

## 1. The Problem: Why the Stack Reconciler Had to Go

### The original architecture

Before React 16, React used what is now called the Stack Reconciler. It did its job well for a long time but as applications grew larger and more interactive, its fundamental design began to show cracks.

The Stack Reconciler rendered updates by recursively traversing the component tree in a single, synchronous pass. Once it started, it could not stop. Every update whether a small text change or a full page re-render, blocked the main thread until completion. There was no way to pause, prioritize, or interrupt rendering work.

This created three concrete problems:

- No way to pause, prioritize, or interrupt rendering mid-way
- Every update, big or small, blocked the main thread entirely
- Deep component trees caused the UI to visibly freeze

### The 16.67ms problem

To understand why this mattered, consider how browsers render frames. Most modern devices target 60fps or 90fps, which means a new frame must be produced approximately every:

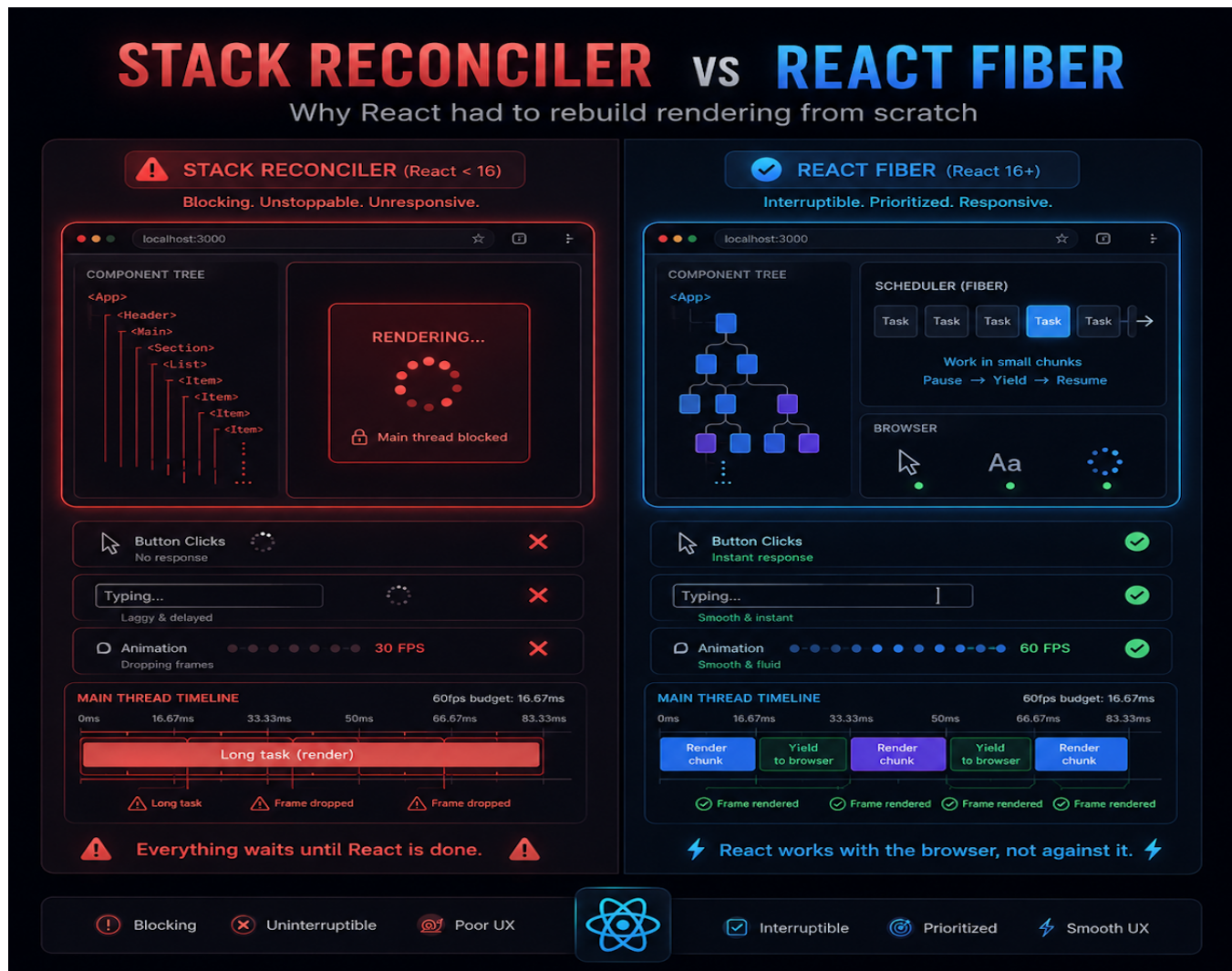
16.67ms for 60fps

11.11ms for 90fps

This is a strict performance budget. React must complete its rendering work within this window and that's before accounting for the time the browser itself needs to perform layout and paint. In practice, React needs to finish significantly earlier than 16.67ms to leave the browser enough room to do its own work.

If the combined time of React rendering plus browser painting exceeds 16.67ms, the browser misses its chance to display the frame. The result is a dropped frame which is visible to the user as a stutter or lag. For static content, this is rarely an issue. But once your app includes animations, frequent updates, or large data-driven lists, staying within the frame budget becomes critical.

The Stack Reconciler had no mechanism to stay within this budget on complex UIs. It traversed the entire component tree for every update, performed synchronous rendering with no support for batching or prioritizing updates, and had no way to split or pause work across frames. These limitations made a complete rewrite of React's core rendering engine inevitable and that rewrite became React Fiber.



## 2. Enter React Fiber: A New Rendering Engine

### What is React Fiber?

React Fiber is the complete rewrite of React's rendering core, introduced in React 16. At its heart, Fiber is a scheduler — an engine that breaks rendering work into small, discrete units called fibers, and manages how those units are processed over time.

The key insight behind Fiber is that rendering does not have to be a single, uninterruptible operation. By breaking work into units, React can process a little, check whether there is time left in the current frame, and either continue or yield control back to the browser. This enables a new class of features that were impossible with the Stack Reconciler:

- Incremental rendering: the UI stays responsive even during large updates
- Pausing and resuming: React can stop mid-render and pick up exactly where it left off
- Priority-based scheduling: user interactions can interrupt background rendering work
- Concurrency primitives... the foundation for Suspense, startTransition, and Concurrent Mode

### What is a Fiber node?

Every element in your React application, every component, every DOM element, every fragment gets its own internal data structure called a fiber node. Think of each fiber as a "worker" assigned to one piece of your UI. Together, they form the Fiber tree, which maps directly to the component hierarchy you write in JSX.

A fiber node carries everything React needs to manage that piece of UI. Here is what lives inside each one:

- **tag:** The internal type of the component. For example, 5 represents a HostComponent like a `<div>`, and 0 represents a FunctionComponent.
- **type:** The actual component or DOM tag React should render: for example "h1", "div", or a reference to your `MyComponent` function.
- **pendingProps:** The props passed to the component during the current render, what React is preparing to use.
- **memoizedProps:** The props that were used during the last committed render. React compares these to `pendingProps` to decide whether re-rendering is necessary.
- **return:** A reference to the parent fiber in the component tree, allowing React to walk back up after finishing a branch.
- **child:** A pointer to the first child fiber, forming a linked list of all children under the current node.
- **sibling:** A pointer to the next sibling fiber at the same level essentially the next JSX element inside the same parent.
- **stateNode:** The actual output of the fiber. For host elements like `<div>`, this is the real DOM node. For class components, it holds the class instance. For function components, it is null because function components have no instance.

These fields together give React everything it needs to be smart about rendering. It can check whether props changed, skip subtrees with no pending work, pause in the middle of a traversal, and resume from exactly the right place, all because each unit of work is a self-contained fiber node.

&gt;&gt; reactFiberNode

```

< ▼ Object { tag: 5, key: null, elementType: "h1", type: "h1", stateNode: h1#h1-id ☞
  _debugHookTypes: null
  _debugInfo: null
  _debugNeedsRemount: false
  _debugOwner: Object { tag: 0, key: null, index: 0, ... }
  _debugStack: Error: react-stack-top-frame
  _debugTask: null
  actualDuration: 2
  actualStartTime: 1551
  alternate: null
  child: null ← First Nested Child
  childLanes: 0
  deletions: null
  dependencies: null
  elementType: "h1"
  flags: 1048576
  index: 0
  key: null
  lanes: 0
  memoizedProps: Object { id: "h1-id", children: "Hello World!!" }
  memoizedState: null
  mode: 27
  pendingProps: Object { id: "h1-id", children: "Hello World!!" }
  ref: null
  refCleanup: null
  return: Object { tag: 5, elementType: "div", type: "div", ... }
  selfBaseDuration: 0
  sibling: Object { tag: 5, elementType: "h2", type: "h2", ... }
  stateNode: <h1 id="h1-id"> ☞ Next Element at Same Level
  subtreeFlags: 0
  tag: 5
  treeBaseDuration: 0
  type: "h1" ← Element Type Info
  updateQueue: null
  <prototype>: Object { ... }
  
```

Props from last completed render

Parent Fiber Node

The Real DOM Node

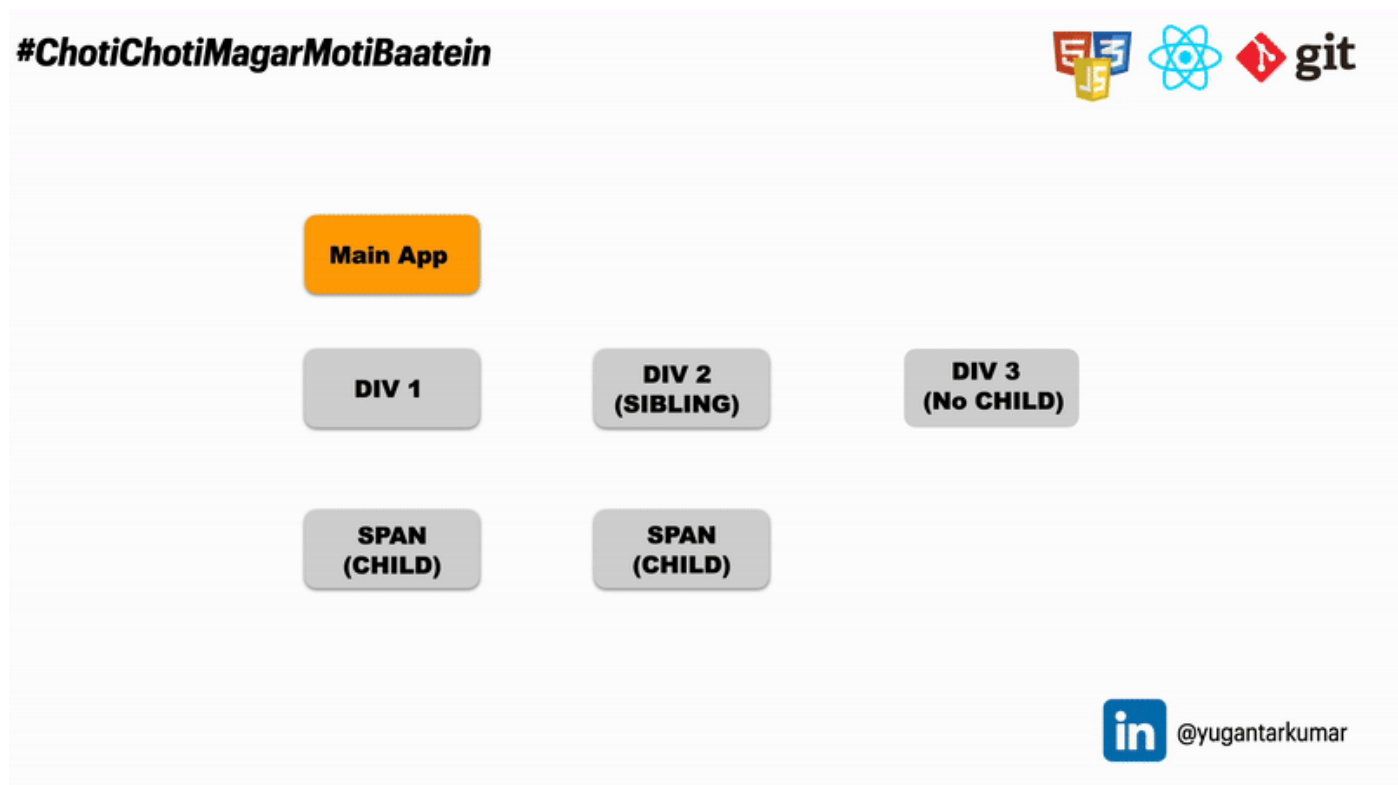


@yugantarkumar



### 3. How React Walks the Tree: Depth-First Traversal

When React needs to reconcile your UI by figuring out what changed and what needs to update, it walks the Fiber tree using a depth-first traversal. This is not new to Fiber; the Stack Reconciler also used depth-first traversal. But Fiber made it interruptible, which is the key difference.



You can also see working gif here: <https://www.linkedin.com/feed/update/urn:li:activity:7463204790299439104/>

#### How the traversal works

React starts at the root of the Fiber tree and follows a consistent pattern:

1. **Start at the Root:** React begins at the Fiber root, the top of the component tree.
2. **Go Deep, Child by Child:** React traverses down through the first child, then its child, and so on, until it reaches the deepest leaf node.
3. **Return, Then Move to Siblings:** After reaching a leaf, React moves back up to the parent, checks for siblings, and dives into that branch next.

This gives React an  $O(n)$  traversal i.e every node is visited exactly once. When node types differ (for example, a `<div>` replaced by a `<span>`), React discards the entire old subtree rather than attempting to reconcile it. For lists, key-based reconciliation allows React to efficiently reorder and reuse nodes rather than recreating them.

#### What Fiber added: interruptibility

Before Fiber, this traversal was all-or-nothing. A large tree could block the browser for tens of milliseconds. Fiber changed this by making the traversal interruptible and incremental:

- **Interruptible:** React can pause mid-traversal to handle a high-priority event like a keystroke, then resume.

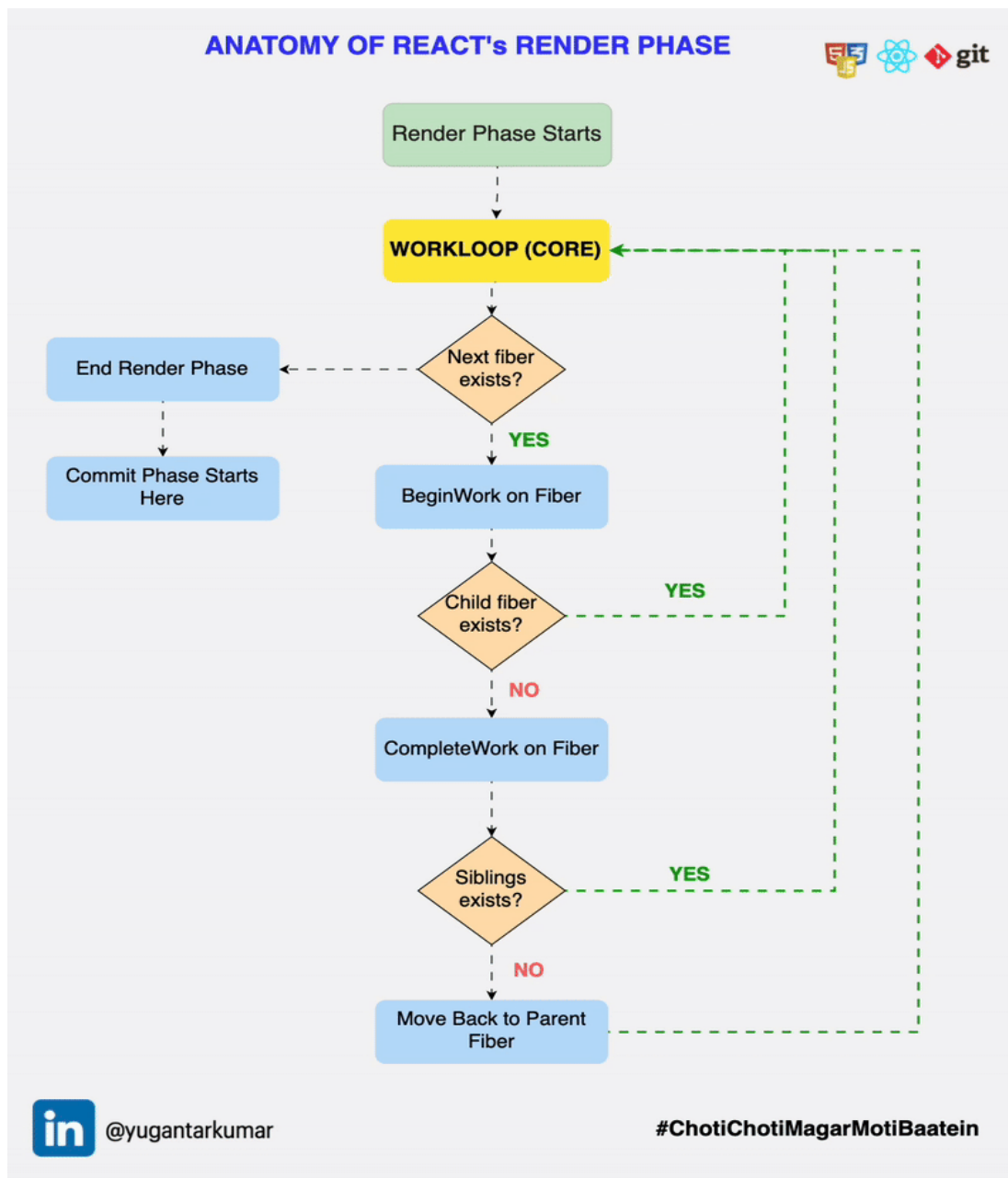
- **Incremental:** When React resumes, it picks up exactly where it stopped, without starting over from the root.

This is possible because the fiber tree is a persistent data structure maintained in memory. React always knows its current position and the next unit of work to process.

## 4. The Render Phase: WorkLoop, beginWork & completeWork

Every re-render in React passes through three distinct stages before anything appears on screen:

- **Render Phase:** React figures out what needs to change (no DOM updates happen here).
- **Commit Phase:** React applies those changes to the actual DOM.
- **Browser Paint:** The browser visually updates the screen.



The Render Phase is where all the intelligent work happens which includes reconciliation, diffing, priority checking. It is also the phase that Fiber made interruptible.

## The WorkLoop

At the heart of the Render Phase is the workLoop. This is the central loop that drives reconciliation. It picks one fiber node at a time and processes it, continuing until either all work is finished or React decides it needs to yield control back to the browser.

Conceptually, you can think of it as:

```
while (nextUnitOfWork !== null && isTimeAvailable === true) {  
  nextUnitOfWork = performUnitOfWork(nextUnitOfWork);  
}
```

Each iteration of this loop processes one fiber node. React does not tackle the entire tree at once, it processes one unit of work, checks if there is still time available in the current frame, and either continues or yields. This is the mechanism that keeps your UI responsive even during large re-renders.

## beginWork

For each fiber the workLoop picks up, it first calls beginWork. This function is responsible for deciding what needs to happen for that fiber:

- On the first render, everything is new, so all child fibers are created from scratch.
- On updates, React compares the new props and state against the previous values.
- If nothing changed on the current fiber, React can reuse the existing child fibers and skip reconciling that entire subtree.
- If something changed, new child fibers are created only for the parts that need updating.

This is exactly where React.memo does its work. When a component is wrapped in React.memo, React checks during beginWork whether the props have changed. If they have not, React skips re-rendering that component entirely, and its entire child subtree is reused as-is. This explains why React.memo can be such an effective optimization as it short-circuits an entire branch of the reconciliation.

It is worth noting that in **React 19**, the React Compiler automatically applies memoization across your component tree. When using the compiler, you no longer need to manually add React.memo, useMemo, or useCallback. The compiler handles this at build time. React.memo itself is still available and still works the same way, but with the compiler enabled it becomes largely unnecessary.

## completeWork

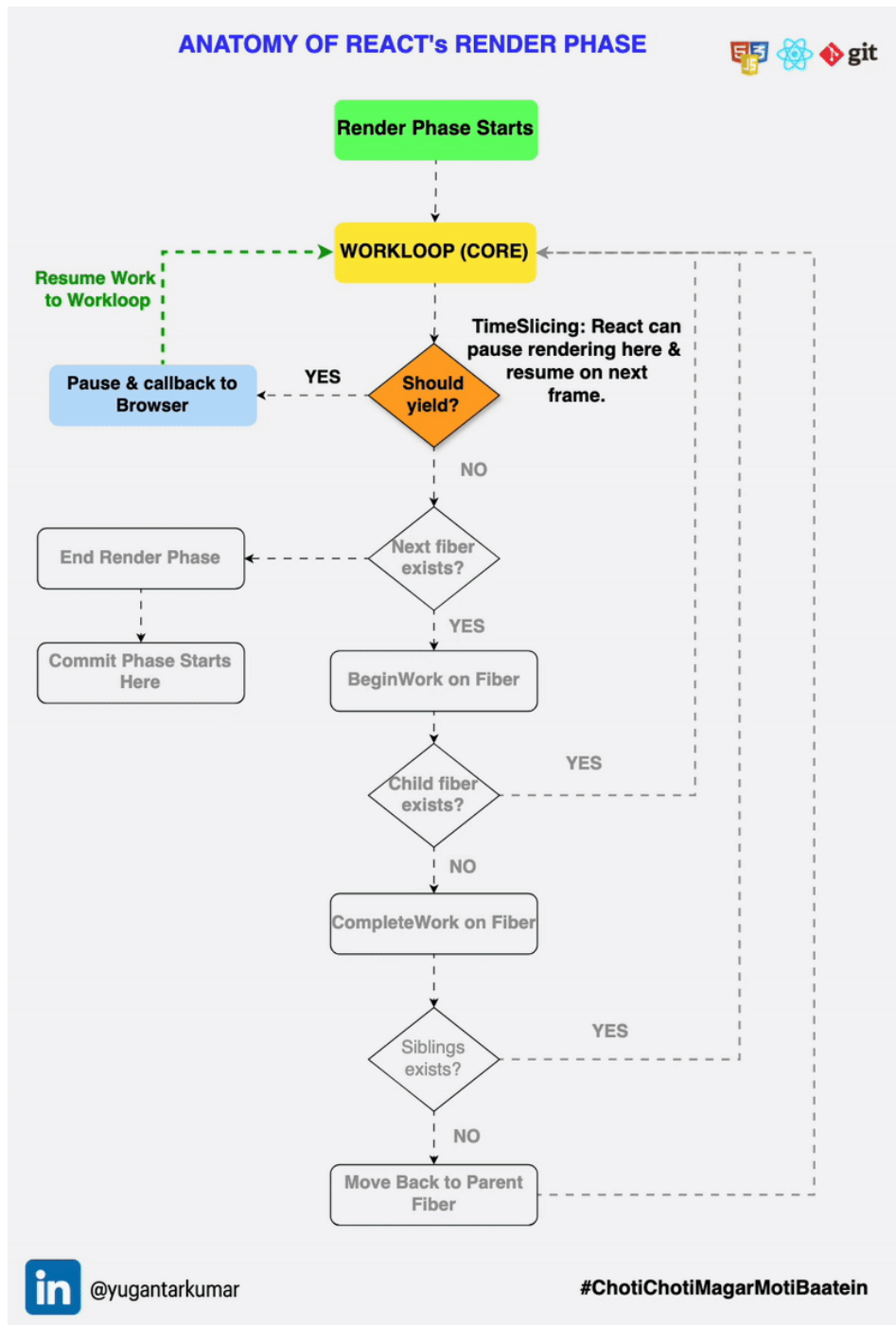
After beginWork finishes processing a fiber, completeWork takes over. This function finalizes the work for that node:

- For host components like <div> or <span>, React creates the actual DOM node in memory and stores a reference to it in fiber.stateNode. Note that this DOM node is not yet attached to the live DOM tree. It exists only in memory at this stage.
- React attaches side-effect flags to the fiber, stored in fiber.flags. These flags include Placement (a new node needs to be inserted), Update (an existing node needs to change), and Deletion (a node needs to be removed), along with flags for refs and hooks like Passive and Layout.
- React builds the Effect List: a linked list of all fibers that carry side effects, which the Commit Phase will walk in order to apply changes. More on this in the next section.

beginWork decides what changes are needed and creates or reuses child fibers. completeWork finalizes the fiber, prepares DOM nodes in memory, and queues side effects. Together, they form the complete backbone of the Render Phase.

## 5. Time Slicing: How React Stays Responsive

You have probably heard the rule that React should finish rendering within 16.67ms for a 60fps screen. But React does not always finish in one go and that is by design. Thanks to Time Slicing, React can pause, yield to the browser, and resume — all without losing its place or corrupting state.





## Before Fiber: blocking rendering

In the Stack Reconciler era, rendering was a single blocking task. Once React started reconciling the component tree, it ran until done. There was no mechanism to pause. If a large tree took 80ms to reconcile, the browser was locked for that entire duration and no user input, no animations, no paint updates.

## How Time Slicing works

Fiber fixes this by breaking rendering into small units of work (fibers) and checking available time after each one. The check happens inside the workLoop via an internal function called `shouldYield()`, which is part of React's Scheduler package:

- **If `shouldYield()` returns false:** React still has time in the current frame, so it continues to the next fiber.
- **If `shouldYield()` returns true:** The time budget is exhausted. React yields control back to the browser so it can paint, handle input events, or run other queued work. On the next available frame, React resumes exactly where it left off.

This transforms React from a blocking, greedy renderer into a cooperative one. It shares the main thread with the browser rather than monopolizing it.

## What this enables

Time Slicing is what makes several React behaviours possible that would otherwise seem contradictory:

- Your app stays responsive during heavy renders because React yields before dropping a frame.
- High-priority updates like typing or clicking can interrupt low-priority background rendering work.
- Animation frames are not dropped because React checks its budget before each fiber, not just before the entire render.

Each fiber node carries two fields called `lanes` and `childLanes` that enable this priority system. These are covered in detail in the next section.

## 6. Lanes & ChildLanes: React's Priority System

Time Slicing explains how React can pause and resume. But how does React decide what to work on first? That is the job of Lanes which is React's internal priority system, introduced to replace the older expiration time model.

#ChotiChotiMagarMotiBaatein

reactLogo git

```
>> reactFiberNode
< ▼ Object { tag: 5, key: null, elementType: "h1", type: "h1", stateNode: h1#h1-id
  _debugHookTypes: null
  _debugInfo: null
  _debugNeedsRemount: false
  ▶ _debugOwner: Object { tag: 0, key: null, index: 0, ... }
  ▶ _debugStack: Error: react-stack-top-frame
  _debugTask: null
  actualDuration: 2
  actualStartTime: 1551
  alternate: null
  child: null
  childLanes: 0
  deletions: null
  dependencies: null
  elementType: "h1"
  flags: 1048576
  index: 0
  key: null
  lanes: 0
  ▶ memoizedProps: Object { id: "h1-id", children: "Hello World!!" }
  memoizedState: null
  mode: 27
  ▶ pendingProps: Object { id: "h1-id", children: "Hello World!!" }
  ref: null
  refCleanup: null
  ▶ return: Object { tag: 5, elementType: "div", type: "div", ... }
  selfBaseDuration: 0
  ▶ sibling: Object { tag: 5, elementType: "h2", type: "h2", ... }
  ▶ stateNode: <h1 id="h1-id">
  subtreeFlags: 0
  tag: 5
  treeBaseDuration: 0
  type: "h1"
  updateQueue: null
  ▶ <prototype>: Object { ... }
```

First Nested Child

childLanes: 0

Props from last completed render

lanes: 0

Parent Fiber Node

The Real DOM Node

Element Type Info

Next Element at Same Level

Lanes & ChildLanes

- Manages Priority of Work
- Internally defines in bitmask

in @yugantarkumar

### Lanes

Every fiber node has a lanes field that represents the priority of pending updates on that specific fiber. Lanes are implemented as bitmasks — integers where each bit corresponds to a different priority level. The smaller the lane value, the more urgent the update.

React defines several built-in lane priorities, from most to least urgent:

- **SyncLane:** Synchronous updates that must happen immediately, like input handling.
- **InputContinuousLane:** Continuous input events like dragging or scrolling.
- **DefaultLane:** Normal updates triggered by setState or data fetching.
- **TransitionLane:** Lower-priority transitions triggered via startTransition.
- **IdleLane:** Work that can be deferred indefinitely until the app is idle.

Using bitmasks means React can check or combine multiple lane priorities with a single bitwise operation which is extremely fast compared to comparing objects or arrays.

## ChildLanes

Alongside lanes, every fiber also has a `childLanes` field. This represents the aggregate of all pending lanes across the fiber's entire subtree — its children, grandchildren, and all descendants.

The significance of `childLanes` is that it propagates upward through the tree. When a fiber deep in the tree gets a pending update, that update's lane is merged into the `childLanes` of every ancestor all the way up to the root. This means that at any point in the tree, a parent fiber can check in  $O(1)$  time whether any of its descendants have pending work without traversing the subtree.

This is what enables `React.memo` to work correctly in the presence of context updates. When a parent component is memoized and its props have not changed, React would normally skip it. But if a child inside that memoized subtree has a pending update (for example, because it reads from a React context that changed), `childLanes` ensures React still dives into that child even though the parent was skipped.

## Batching via Lanes

Lanes also power React's automatic batching behaviour. When two `setState` calls happen inside the same event handler, they are assigned the same lane. React batches them together into a single render pass rather than triggering two separate re-renders. If two updates land in different lanes, for example, one synchronous and one deferred via `startTransition`, React can run the urgent one immediately and schedule the other for later.

The result is a rendering system that is both fast and intelligent: it skips subtrees with no pending work, batches updates of the same priority, and prioritizes urgent work while deferring less important updates — all without the developer needing to think about it explicitly.

## 7. The Commit Phase: Making It Real

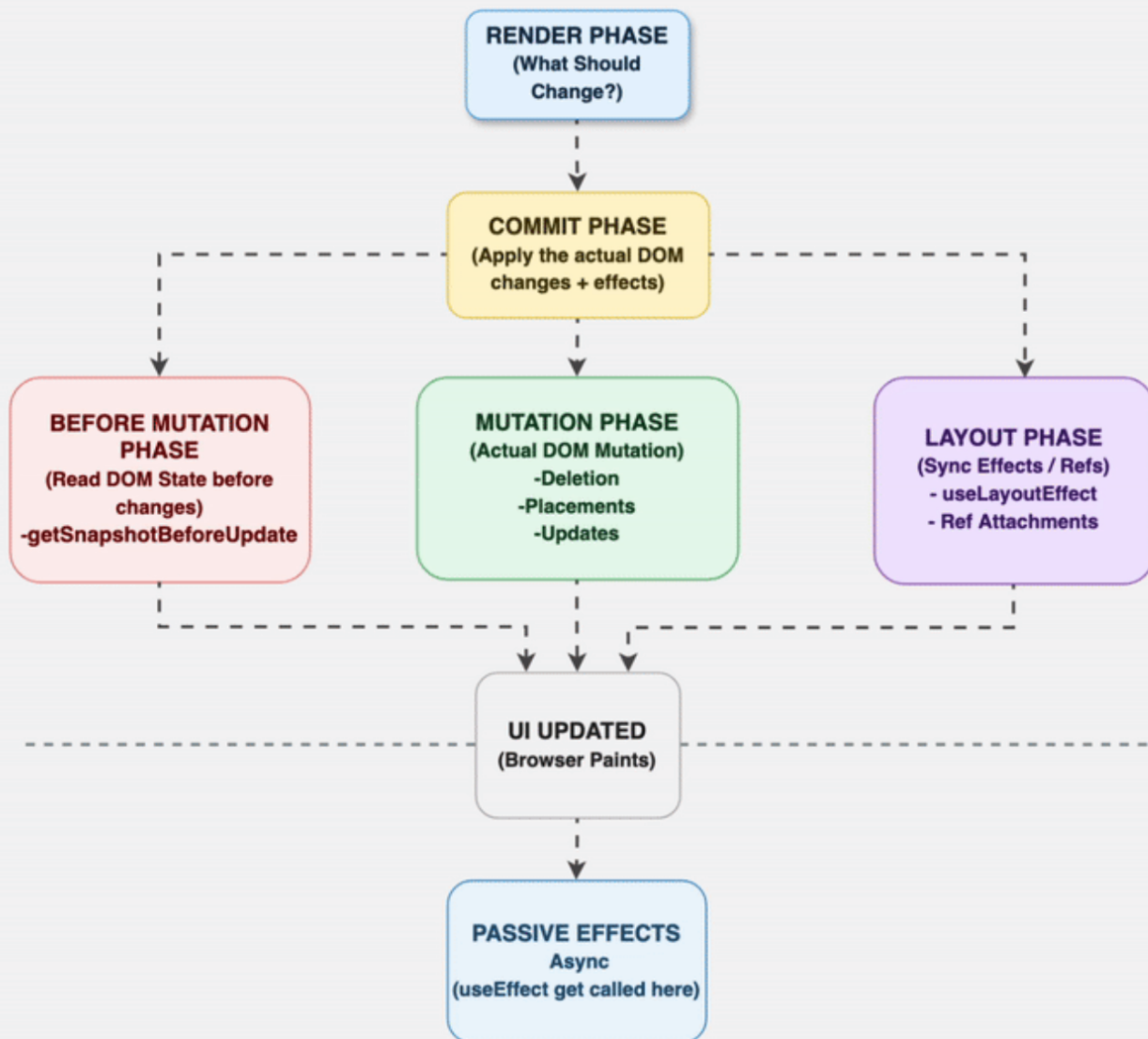
The Render Phase decides what needs to change. The Commit Phase makes it happen. This is where the DOM is actually mutated, refs are attached, and effect callbacks are scheduled or executed.

### Key characteristics of the Commit Phase

Unlike the Render Phase, the Commit Phase is synchronous and blocking. Once it begins, React cannot pause or interrupt it. It must finish applying all DOM updates before the browser can paint again. This is intentional as partial DOM updates would leave the UI in an inconsistent state visible to the user.

Because the Commit Phase is blocking, the work that happens here must be fast and carefully ordered. Heavy synchronous work in the Commit Phase directly causes dropped frames.

# React's Commit Phase



## The four stages of the Commit Phase

The Commit Phase is divided into four stages, each serving a distinct purpose:

### Before Mutation

React's last opportunity to read the current DOM before any changes are made. Components can capture information like scroll positions or element dimensions that would be lost once the DOM updates. This is where `getSnapshotBeforeUpdate` runs.

### Mutation Phase

The actual DOM changes happen here i.e. deletions, insertions, and updates are applied in that specific order. React walks the fiber nodes in depth-first post-order traversal (children before parents), executing each flagged mutation.

## Layout Phase

Runs immediately after DOM mutations but before the browser paints. Refs are attached to their DOM nodes, and `useLayoutEffect` callbacks fire synchronously. This phase is blocking.

## Passive Effects

After the browser has painted, React runs `useEffect` callbacks asynchronously via its Scheduler. This is non-blocking and does not delay the user from seeing the updated UI.

---

# 8. Before Mutation: Capturing the DOM Snapshot

Before React makes any changes to the DOM, there is one last chance to read its current state. This window is called the Before Mutation Phase, and it exists for a precise reason: certain information like scroll positions, element sizes, and layout measurements is permanently lost once the DOM is updated.

## `getSnapshotBeforeUpdate`

The primary function that runs during this phase is `getSnapshotBeforeUpdate`, a class component lifecycle method. It receives the previous props and state and can return any value, which React then passes as the third argument to `componentDidUpdate`.

```
getSnapshotBeforeUpdate(prevProps, prevState) {  
  const scroll = this.listRef.scrollTop;  
  return scroll;  
}  
  
componentDidUpdate(prevProps, prevState, snapshot) {  
  // snapshot is the scroll value captured before mutation  
}
```

## `getSnapshotBeforeUpdate(prevProps, prevState)`

If you implement `getSnapshotBeforeUpdate`, React will call it immediately before React updates the DOM. It enables your component to capture some information from the DOM (e.g. scroll position) before it is potentially changed. Any value returned by this lifecycle method will be passed as a parameter to `componentDidUpdate`.

For example, you can use it in a UI like a chat thread that needs to preserve its scroll position during updates:

```
class ScrollingList extends React.Component {  
  constructor(props) {  
    super(props);  
    this.listRef = React.createRef();  
  }
```

```
  getSnapshotBeforeUpdate(prevProps, prevState) {  
    // Are we adding new items to the list?  
    // Capture the scroll position so we can adjust scroll later.  
    if (prevProps.list.length < this.props.list.length) {  
      const list = this.listRef.current;  
      return list.scrollHeight - list.scrollTop;  
    }  
    return null;  
  }
```

`getSnapshotBeforeUpdate` returns value which appears as 3rd argument in `componentDidUpdate`

```
  componentDidUpdate(prevProps, prevState, snapshot) {  
    // If we have a snapshot value, we've just added new items.  
    // Adjust scroll so these new items don't push the old ones out of view.  
    // (snapshot here is the value returned from getSnapshotBeforeUpdate)  
    if (snapshot !== null) {  
      const list = this.listRef.current;  
      list.scrollTop = list.scrollHeight - snapshot;  
    }
```

## A real-world example

The React documentation gives a compelling example of this in action: a chat UI where new messages are continuously added at the bottom. When new messages arrive and the DOM updates, the scroll position can jump unexpectedly, pushing the user away from where they were reading.

Using `getSnapshotBeforeUpdate`, you capture the user's current scroll position just before the DOM changes. After the update, `componentDidUpdate` receives that captured value and can restore the scroll position, making the experience feel natural and continuous regardless of how many messages were added.

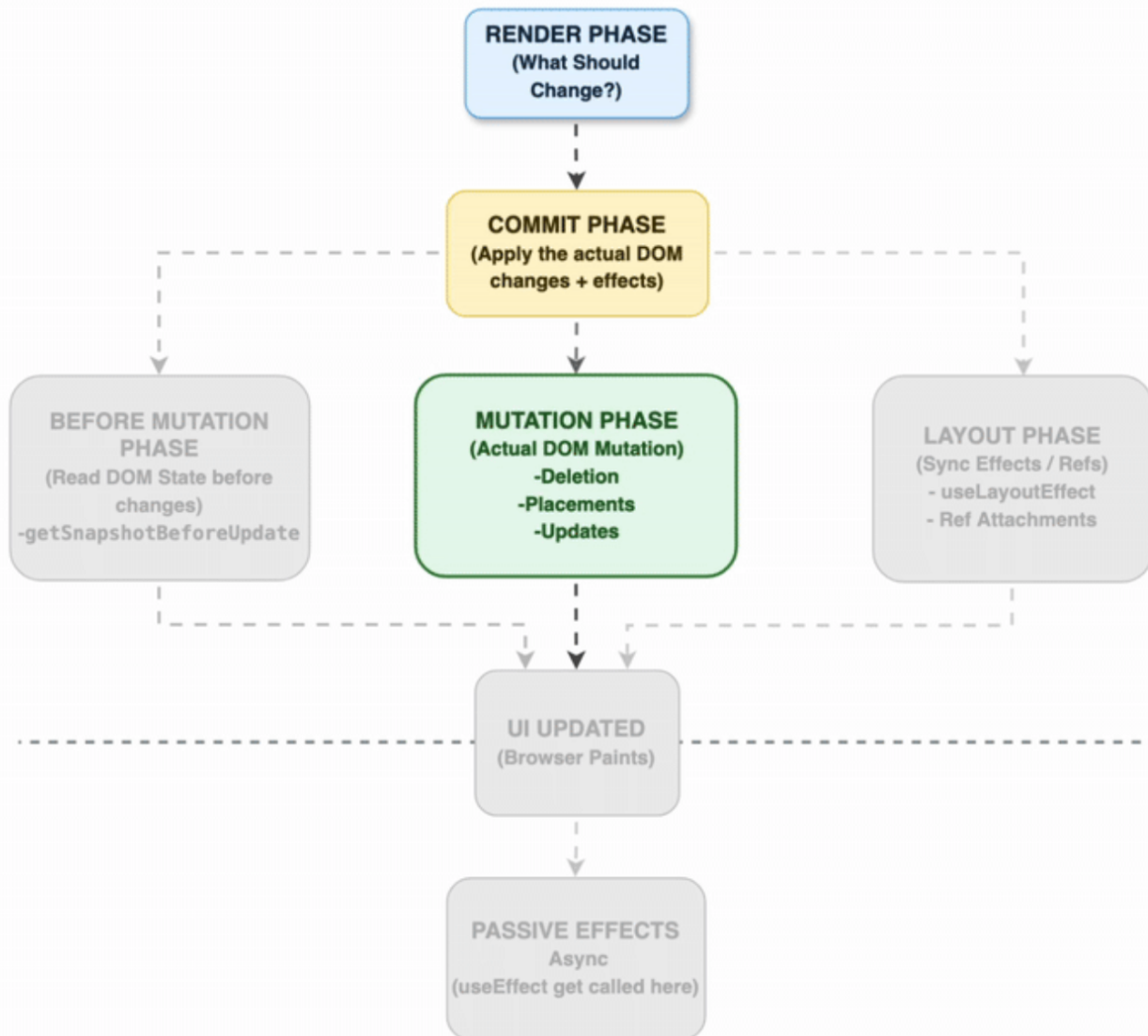
This lifecycle has no direct hook equivalent. For function components that need similar pre-mutation DOM reading, the recommendation as of React 19 is to use `useRef` to track the DOM value and `useLayoutEffect` to restore it after the update.



## 9. Mutation Phase: Deletions, Placements & Updates

The Mutation Phase is where React's carefully planned changes finally meet the real DOM. Everything the Render Phase prepared, every Placement, Update, and Deletion flag gets applied here in a predictable, synchronous sequence.

### React's Commit Phase (Mutation)



@yugantarkumar

#ChotiChotiMagarMotiBaatein

### Why the order matters

React applies mutations in a strict three-step order, and this ordering is not arbitrary:

#### 1. Deletions

Old DOM nodes are removed first. This happens before insertions for three important reasons:

- It prevents duplicate key or ID conflicts. If an element with the same key is being replaced, the old one must be removed before the new one can be inserted.
- It clears the position so new nodes can be inserted cleanly, avoiding complex repositioning logic.
- It runs cleanup: event listeners are detached and refs are detached, preventing memory leaks from removed elements.

## 2. Placements

Newly created DOM nodes are inserted next. At this point the parent DOM nodes already exist, the deleted nodes are gone, and React can insert new elements into the correct positions without interference.

## 3. Updates

Finally, existing DOM nodes are updated with text changes, attribute modifications, style changes, and event listener adjustments. Updates run last so React is always operating against a stable, fully-formed DOM structure.

## Traversal order

During the Mutation Phase, React walks the fiber nodes in depth-first post-order meaning child nodes are always mutated before their parents. This guarantees DOM consistency, predictable layout behaviour, and correct execution order for the effects that follow.

React knows exactly which operations to perform because of the flags set during the Render Phase. The Mutation Phase simply reads those flags and executes the corresponding DOM operations in order.

---

# 10. Layout Phase: Before the User Sees Anything

After the DOM is fully updated but before the browser paints, React has one final synchronous window to interact with the updated DOM. This is the Layout Phase, and it exists precisely because some operations must measure or adjust the DOM before the user sees it.

## What happens in the Layout Phase

Two things happen during the Layout Phase, in this order:

### Ref attachments

Refs are attached to their corresponding DOM nodes. After this step, `ref.current` reliably points to the correct, updated DOM element. This is why you should never try to read a ref during the Render Phase, the DOM might not exist yet. The Layout Phase is when `ref.current` becomes accurate.

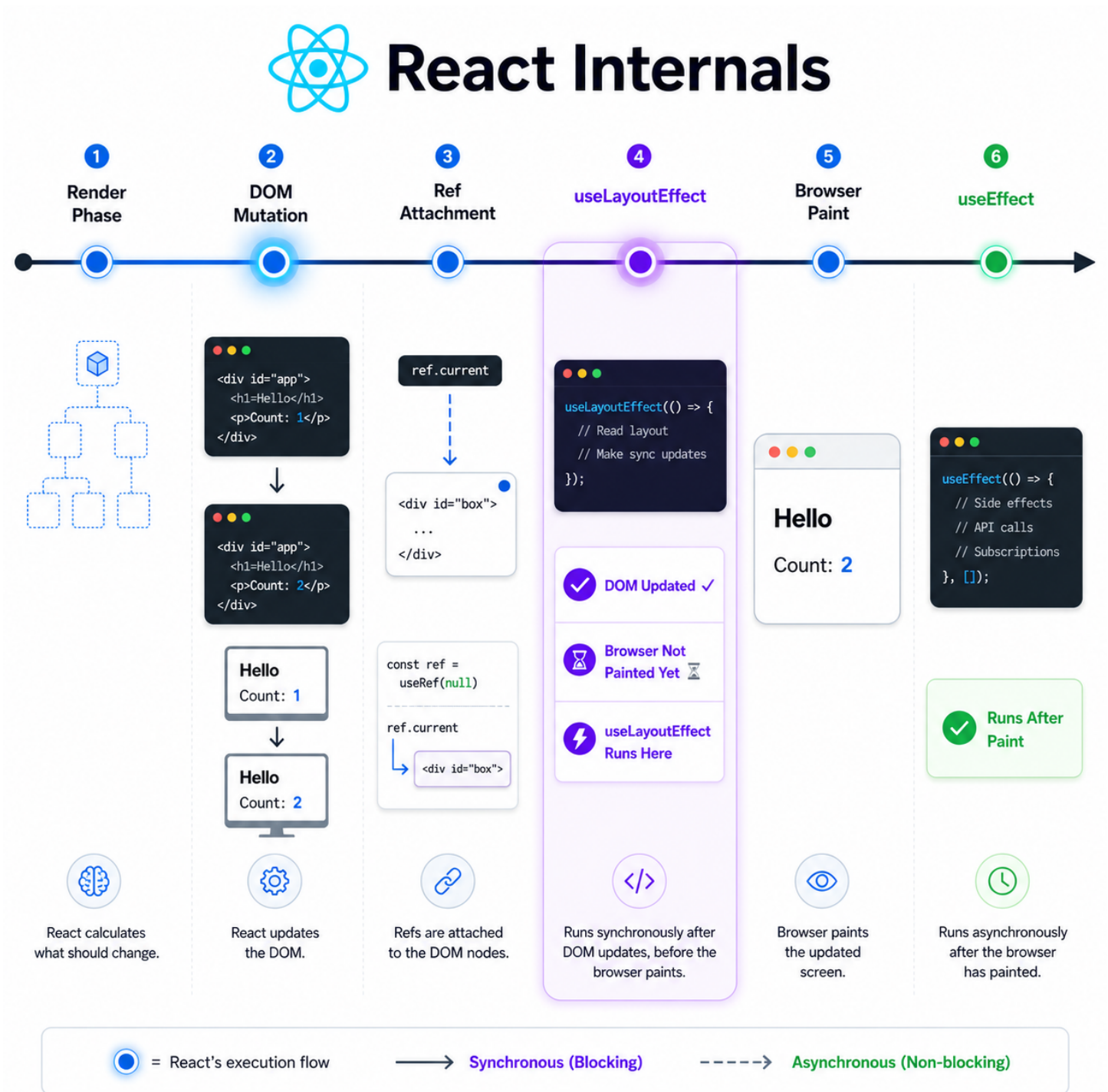
### useLayoutEffect callbacks

`useLayoutEffect` cleanup functions run first (from the previous render), followed by the new `useLayoutEffect` callbacks. These effects are synchronous and run before the browser paints, which is the defining characteristic of `useLayoutEffect`.

This timing makes `useLayoutEffect` appropriate for:

- DOM measurements (reading element dimensions or positions after they have been updated)
- Layout calculations that depend on the new DOM state

- Imperative DOM updates that must be invisible to the user, for example, setting a position or size before paint to avoid a flash of incorrectly-positioned content



## The Layout Phase is blocking

Like the Mutation Phase, the Layout Phase cannot be paused or interrupted. It must finish before the browser paints. This means heavy work inside `useLayoutEffect` directly delays when the user sees the updated screen which is why `useLayoutEffect` should be used only when its synchronous timing is genuinely necessary. For the vast majority of side effects (data fetching, subscriptions, logging), `useEffect` is the right choice.

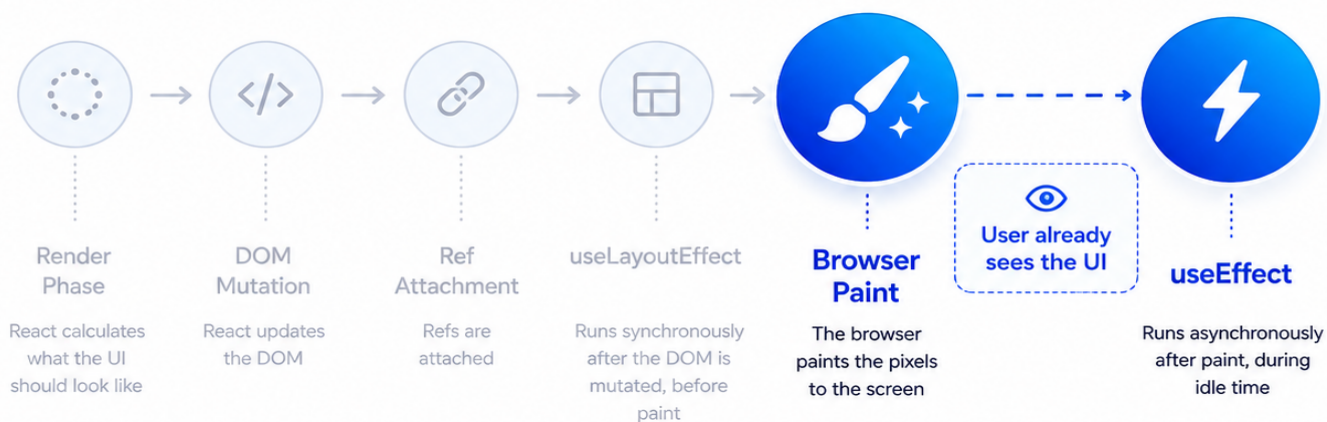
Layout effects run in child-first, parent-last order — the same post-order traversal used throughout the Commit Phase. This ensures that children complete their layout work before parents try to measure or adjust based on them.

## 11. Passive Effects: Why `useEffect` Runs After Paint

The Commit Phase has finished. The DOM is updated. The browser has painted the new UI. And yet React is still not done. After all of this, React runs one more round of work i.e. the Passive Effects Phase. This is where `useEffect` runs.



# The Difference Between `useLayoutEffect` and `useEffect`



Before Paint (Not Visible)

After Paint (Visible)



### `useLayoutEffect`

Blocks the browser from painting. Use for layout reads, measurements, and DOM mutations that must happen before the user sees anything.



### `useEffect`

Does not block paint. Use for non-visual side effects like data fetching, subscriptions, logging, and analytics.



## What are Passive Effects?

Passive Effects are all the side effects flagged as Passive during the Render Phase, primarily useEffect callbacks. Unlike the earlier phases of the Commit Phase, Passive Effects are asynchronous and non-blocking. They run after the browser has already painted the screen.

The execution order mirrors the structure of the earlier phases:

- First, useEffect cleanup functions from the previous render run (for any fiber whose deps changed).
- Then, the new useEffect callbacks run.

## Why the delay is intentional

The asynchronous timing of useEffect is a deliberate design choice, not a limitation. React's primary goal is to show the user an updated UI as quickly as possible. By deferring effect callbacks until after paint, React ensures that data fetching, subscriptions, event listener registrations, and other side effects never delay what the user sees on screen.

If you think about the most common useEffect uses — fetching data from an API, setting up a WebSocket subscription, logging analytics, syncing with localStorage — none of them need to block the user from seeing the updated UI. Running them after paint is strictly better for perceived performance.

## How React schedules Passive Effects

React uses its internal Scheduler to queue Passive Effects as low-priority work. They run when the main thread is free, after the browser has had the opportunity to handle any pending user interactions or paint updates. This means Passive Effects never interfere with rendering or user responsiveness.

This scheduling is what explains a common React interview question: "Why does useEffect sometimes run before I expect it to in development mode?" In React 19 with Strict Mode enabled in development, React intentionally runs effects twice i.e. mounting, unmounting, and remounting to help surface bugs where cleanup functions are missing or incorrect. This double-invocation happens only in development and does not reflect production behaviour.

---

## 12. Putting It All Together

You now have a complete picture of how React works under the hood, from the moment a re-render is triggered to the moment the user sees the updated screen. Here is the full journey in summary:

### The rendering pipeline

**Trigger:** A setState call, a context update, a parent re-render, or an external store change causes React to schedule work.

**Render Phase (interruptible):** React builds or updates the Fiber tree using a depth-first traversal. The workLoop processes one fiber at a time via beginWork and completeWork. Time Slicing allows React to yield to the browser between fibers when the frame budget is running low. Lanes and childLanes ensure high-priority work runs first and clean subtrees are skipped.

**Commit Phase (blocking):** Runs synchronously and cannot be interrupted. It has four sub-phases: Before Mutation (snapshot capture via `getSnapshotBeforeUpdate`), Mutation (actual DOM deletions, insertions, and updates), Layout (ref attachments and `useLayoutEffect`), and Passive Effects (`useEffect`, running asynchronously after paint).

**Browser Paint:** After the Mutation and Layout phases complete, the browser renders the updated pixels to screen. Passive Effects run after this point.

## Key mental models to carry forward

- **The Render Phase is pure and side-effect-free.** It can be paused, restarted, or discarded. Never put DOM mutations or side effects in rendering logic.
- **The Commit Phase is synchronous and final.** Once it starts, it cannot stop. Keep `useLayoutEffect` callbacks lean.
- **`useEffect` is intentionally late.** It runs after paint by design, making it safe for I/O, subscriptions, and analytics without blocking the UI.
- **`React.memo` and the React Compiler both operate during `beginWork`.** They work by allowing React to short-circuit reconciliation for fibers whose inputs have not changed.
- **Lanes are priorities, not sequential steps.** Multiple updates can be in-flight simultaneously, each assigned to a lane. React picks the highest-priority lane to process first.

Understanding React's internals does not just satisfy curiosity but it changes how you write code. You start reasoning about when effects run, why certain optimizations work, what makes a component expensive, and how concurrent features like `Suspense` and `startTransition` are able to exist. The Fiber architecture is the foundation that makes modern React possible, and now you understand exactly how it works.

---

— End —

## What's Coming Next

React Fiber is a vast topic and this guide covers the most important concepts every senior React developer should know. Here's what's planned for upcoming editions:

- Reconciliation deep dive - how React diffs two fiber trees in detail
- `Suspense` internals - how React pauses rendering and shows fallbacks
- React Compiler internals - what the compiler actually does to your code
- Concurrent features - how `startTransition` and `useDeferredValue` work under the hood
- Error boundaries - how React catches and recovers from render errors

*This guide is updated as new editions are released. You'll always have access to the latest version at the same link.*

**Follow Me on LinkedIn.** <https://www.linkedin.com/in/yugantar-kumar/>



